

Two-Machine Flow Shop Scheduling to Minimize Total Weighted Completion Time: Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

We study the two-machine flow shop scheduling problem to minimize the total weighted completion time, denoted as $F2 \parallel \sum W_j C_j$. By partitioning jobs into at most K categories according to their weights in non-increasing order, the schedule decomposes into K sequential segments. We establish four structural properties of optimal schedules: the permutation property, the positional constraint $\pi(J_k) \leq k$, the block-end condition requiring the last job of segment k to carry weight W_k , and the SPT(1)-LPT(2) ordering rule within each segment. Building on these properties and drawing on the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a pseudo-polynomial time dynamic programming algorithm that constructs the schedule block by block. We further discuss how the algorithm can be converted into a polynomial-time approximation scheme (PTAS) via geometric rounding of weights and processing times.

Keywords: Flow shop scheduling; Total weighted completion time; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Problem Formulation

3 Structural Properties

We partition the job set J into K weight categories $J_1, J_2, \dots, J_K$ such that all jobs in J_k share the same weight W_k , and the weights satisfy $W_1 > W_2 > \dots > W_K$. Let $n_k = |J_k|$ and $\sum_{k=1}^K n_k = n$.

The schedule is divided into K contiguous logical segments (blocks) $S_1, S_2, \dots, S_K$, processed sequentially on both machines. We use $\pi(j)$ to denote the position (block index) to which job j is assigned.

Lemma 3.1 (*Permutation property.*) *For the $F2 \parallel \sum W_j C_j$ problem, there exists an optimal schedule in which the processing order of jobs is identical on both machines.*

This is a classic result for two-machine flow shops with regular objective functions ?.

Lemma 3.2 (*Positional constraint.*) *In an optimal schedule, $\pi(J_k) \leq k$ for all $k = 1, \dots, K$. That is, every job from weight class J_k must be assigned to one of the first k blocks $S_1, S_2, \dots, S_k$.*

Lemma 3.2 implies that after block S_k is processed, all jobs with weights $W_1, \dots, W_k$ are fully scheduled. Consequently, block S_k can only contain jobs from classes $J_k, J_{k+1}, \dots, J_K$.

Lemma 3.3 (*Block-end condition.*) *In an optimal schedule, the last job of block S_k must have weight W_k , i.e., belongs to class J_k , for each $k = 1, \dots, K$.*

Lemma 3.4 (*Intra-block ordering.*) *Within each block S_k , all jobs are sequenced according to the SPT(1)-LPT(2) rule (Johnson's rule): for any two jobs $x, y \in S_k$, job x precedes job y if and only if*

$$\min\{a_x, b_y\} < \min\{a_y, b_x\},$$

or, in case of equality, ties are broken by non-decreasing a_j .

Collectively, these four properties fully determine the structure of an optimal schedule: the jobs are organized into K blocks in order of decreasing weight; within each block, the sequence obeys Johnson's rule; class k jobs cannot appear beyond block k ; and the final position of block k is occupied by a class k job.

4 Dynamic Programming Algorithm

The structural properties established in Section 3 allow us to design a dynamic programming algorithm that builds the schedule block by block. Our DP is inspired by the interval-based state representation technique used in the PTAS for $F2|r_j|C_{\max}$?, where the schedule is partitioned into intervals, jobs are sequenced within each interval by Johnson's rule, and the DP state tracks the completion time of each interval. We adapt this framework to the weighted completion time objective by processing blocks in order and accumulating the weighted sum incrementally. We present the algorithm in four stages, introducing notation at each stage as it becomes relevant.

4.1 Preprocessing

Notation.	n	total number of jobs
	K	number of distinct weight classes
	J_k	set of jobs with weight W_k ($k = 1, \dots, K$)
	W_k	common weight of jobs in J_k , with $W_1 > W_2 > \dots > W_K$
	n_k	$ J_k $, number of jobs in class k
	a_j	processing time of job j on machine 1
	b_j	processing time of job j on machine 2
	w_j	weight of job j (equals W_k iff $j \in J_k$)
	L_k	list of jobs in J_k sorted by Johnson's rule
	S_k	the k -th contiguous block in the schedule

We first partition the n jobs into K sets $J_1, J_2, \dots, J_K$ by weight, with $W_1 > W_2 > \dots > W_K$. Within each set J_k , we sort the jobs according to the SPT(1)-LPT(2) rule (Johnson's order). Let the resulting ordered list be

$$L_k = (j_{k,1}, j_{k,2}, \dots, j_{k,n_k}), \quad k = 1, 2, \dots, K,$$

where $n_k = |J_k|$. Since block S_k of the schedule will contain jobs drawn from classes $k, k + 1, \dots, K$ and must obey Johnson ordering internally, taking prefixes of each L_m ($m \geq k$) and merging them by the same Johnson comparator yields the correct within-block sequence. Algorithm 1 formalizes this merging procedure.

Algorithm 1 Merge.By.Johnson(T): merge jobs into Johnson order

- 1: **Input:** a set of jobs T , each with processing times (a_j, b_j)
 - 2: **Output:** sequence σ in Johnson order
 - 3: Partition T into $T_1 = \{j \in T \mid a_j \leq b_j\}$ and $T_2 = \{j \in T \mid a_j > b_j\}$
 - 4: Sort T_1 by non-decreasing a_j
 - 5: Sort T_2 by non-increasing b_j
 - 6: **return** concatenation of T_1 followed by T_2
-

4.2 State Definition

Notation.	$\mathcal{Y}^{(k)}$	set of non-dominated states after processing blocks 1.. k ($k = 0, \dots, K$)
	$\vec{p}$	prefix vector $(p_{k+1}, \dots, p_K)$; $p_m = \#$ jobs already taken from L_m
	A	total M1 processing time accumulated over blocks 1.. k
	C	M2 completion time at the end of block k (M2 ready for block $k+1$)
	z	total weighted completion time accumulated over blocks 1.. k
	$\vec{0}$	the all-zero vector $(0, \dots, 0) \in \mathbb{N}^K$

For each block index $k = 0, 1, \dots, K$, we maintain a state set $\mathcal{Y}^{(k)}$. A state in $\mathcal{Y}^{(k)}$ represents a complete assignment of jobs to the first k blocks and is described by the tuple

$$(\vec{p}, A, C, z),$$

where:

- $\vec{p} = (p_{k+1}, p_{k+2}, \dots, p_K)$ with $0 \leq p_m \leq n_m$: the number of jobs already taken from class J_m ($m > k$) and assigned to blocks $1, \dots, k$.
- A : the total processing time accumulated on machine 1 over blocks $1, \dots, k$.
- C : the completion time of the last job of block k on machine 2 (i.e., the time at which machine 2 becomes available for block $k+1$). For $k = 0$, $C = 0$.
- z : the total weighted completion time contributed by all jobs assigned to blocks $1, \dots, k$.

By Lemma 3.2, all jobs of classes $1, \dots, k$ must be fully assigned by the end of block k . Hence $\vec{p}$ does not need to track indices $m \leq k$; we have the invariant $p_m = n_m$ for all $m \leq k$ in every state of $\mathcal{Y}^{(k)}$.

The initial state set contains a single entry:

$$\mathcal{Y}^{(0)} = \{(\vec{0}, 0, 0, 0)\}, \quad \text{where } \vec{0} = (0, 0, \dots, 0) \in \mathbb{N}^K.$$

4.3 Computing a Block

$\vec{d}$	selection $(d_{k+1}, \dots, d_K)$: d_m jobs taken from L_m for block $k+1$
r	$= n_{k+1} - p_{k+1}$, number of remaining class $k+1$ jobs (≥ 1)
d_{k+1}	fixed to r : all remaining class $k+1$ jobs must enter this block
T	set of jobs assigned to the current block
σ	T in Johnson order (Algorithm 1)
$A_{\text{init}}, C_{\text{init}}$	M1 total and M2 time at block entry ($= A, C$)
Notation. A_{cur}	running M1 cumulative time across σ
C_{cur}	running M2 completion time across σ
C_j	M2 completion time of job $j \in \sigma$
Δz	weighted cost $\sum_{j \in T} w_j C_j$ from this block
A', C'	M1 total and M2 time after the block (output)
$\vec{p}'$	prefix vector after the block (output)
z'	$z + \Delta z$, accumulated cost (output)

Given a state $(\vec{p}, A, C, z) \in \mathcal{Y}^{(k)}$ and a selection of jobs to populate block S_{k+1} , we must compute the resulting new state. Let the job selection be specified by taking the next d_m jobs from each prefix $L_m[p_m+1 \dots p_m+d_m]$ for $m = k+1, \dots, K$, where $d_{k+1} = r$ (all r remaining class $k+1$ jobs are mandatory by Lemma 3.2) and $d_m \geq 0$ for $m > k+1$.

Let T be the set of selected jobs. Sort T by Johnson's rule (Algorithm 1). To satisfy Lemma 3.3, the last job in the resulting sequence σ must belong to J_{k+1} ; otherwise this selection is discarded.

Starting from initial machine availabilities $(A_{\text{init}}, C_{\text{init}}) = (A, C)$, we process σ through the two-machine flow shop:

$$A_{\text{cur}} \leftarrow A_{\text{init}}, \quad C_{\text{cur}} \leftarrow C_{\text{init}}, \quad \Delta z \leftarrow 0;$$

for each job $j \in \sigma$ in order:

$$A_{\text{cur}} \leftarrow A_{\text{cur}} + a_j, \tag{1}$$

$$C_j \leftarrow \max\{C_{\text{cur}}, A_{\text{cur}}\} + b_j, \tag{2}$$

$$\Delta z \leftarrow \Delta z + w_j \cdot C_j, \tag{3}$$

$$C_{\text{cur}} \leftarrow C_j.$$

After processing all jobs in T , the new state components are:

$$A' = A_{\text{cur}}, \quad C' = C_{\text{cur}}, \quad z' = z + \Delta z,$$

and $\vec{p}'$ is obtained from $\vec{p}$ by incrementing $p_m \leftarrow p_m + d_m$ for each $m > k$.

Algorithm 2 summarizes this procedure.

Algorithm 2 Compute_Block($k, \vec{p}, A, C, \vec{d}$)

Require: Block index k , prefix counts $\vec{p}$, M1 total A , M2 time C , selection vector $\vec{d} = (d_{k+1}, \dots, d_K)$

Ensure: New state $(A', C', \Delta z)$, or INVALID if block-end condition is violated

```
1:  $T \leftarrow \emptyset$ 
2: for  $m = k+1$  to  $K$  do
3:   for  $i = 1$  to  $d_m$  do
4:      $T \leftarrow T \cup \{L_m[p_m + i]\}$ 
5:   end for
6: end for
7:  $\sigma \leftarrow \text{MERGE\_BY\_JOHNSON}(T)$ 
8: if  $\text{weight}(\sigma[|\sigma|]) \neq W_{k+1}$  then
9:   return INVALID
10: end if
11:  $A_{\text{cur}} \leftarrow A, C_{\text{cur}} \leftarrow C, \Delta z \leftarrow 0$ 
12: for  $j \in \sigma$  (in sequence order) do
13:    $A_{\text{cur}} \leftarrow A_{\text{cur}} + a_j$ 
14:    $C_j \leftarrow \max(C_{\text{cur}}, A_{\text{cur}}) + b_j$ 
15:    $\Delta z \leftarrow \Delta z + w_j \cdot C_j$ 
16:    $C_{\text{cur}} \leftarrow C_j$ 
17: end for
18: return  $(A_{\text{cur}}, C_{\text{cur}}, \Delta z)$ 
```

4.4 State Transition

Notation. All symbols $\vec{p}, A, C, z, \vec{d}, r, \Delta z, A', C', \vec{p}', z'$ inherit their meanings from the preceding subsections. Additionally:

$A_{\text{max}} \mid \sum_{j=1}^n a_j$, upper bound on M1 total time
 $C_{\text{max}} \mid \sum_{j=1}^n (a_j + b_j)$, upper bound on M2 completion time

The DP proceeds from $k = 0$ to $K - 1$. For each state $(\vec{p}, A, C, z) \in \mathcal{Y}^{(k)}$, we consider the transition to block $k + 1$.

Let $r = n_{k+1} - p_{k+1}$ be the number of remaining jobs in class J_{k+1} . By Lemma 3.2, these must all enter block $k + 1$, so $d_{k+1} = r$ is fixed ($r \geq 1$). For the higher-indexed classes $m = k+2, \dots, K$, we choose $d_m \in \{0, 1, \dots, n_m - p_m\}$ additional jobs to include in block $k + 1$.

For each admissible vector $\vec{d} = (r, d_{k+2}, \dots, d_K)$, we invoke Algorithm 2. If it returns a valid tuple $(A', C', \Delta z)$, we update:

$$\vec{p}' = (p_{k+2} + d_{k+2}, \dots, p_K + d_K), \quad z' = z + \Delta z,$$

and insert $(\vec{p}', A', C', z')$ into $\mathcal{Y}^{(k+1)}$. If multiple transitions reach the same state $(\vec{p}', A', C')$, only the one with minimal z' is retained.

4.5 Complete Algorithm

Notation. Z^* | optimal (minimum) total weighted completion time

The overall procedure is given in Algorithm 3.

Algorithm 3 DP_F2_WeightedSum(J, A, B, W)

```

1: Input: job set  $J$  with processing times  $a_j, b_j$  and weights  $w_j$ 
2: Output: optimal weighted sum  $Z^*$ 
3: // — Preprocessing —
4: Group jobs by weight into  $J_1, \dots, J_K$  with  $W_1 > \dots > W_K$ 
5: for  $k = 1$  to  $K$  do
6:    $L_k \leftarrow \text{MERGE\_BY\_JOHNSON}(J_k)$  ▷ sort class  $k$  by Johnson
7: end for
8: // — Initialization —
9:  $\mathcal{Y}^{(0)} \leftarrow \{(\vec{0}, 0, 0, 0)\}$  ▷  $\vec{0} \in \mathbb{N}^K$ 
10: // — Block-by-block DP —
11: for  $k = 0$  to  $K - 1$  do
12:    $\mathcal{Y}^{(k+1)} \leftarrow \emptyset$ 
13:   for each  $(\vec{p}, A, C, z) \in \mathcal{Y}^{(k)}$  do
14:      $r \leftarrow n_{k+1} - p_{k+1}$  ▷ remaining class  $k+1$  jobs
15:     for each  $\vec{d} = (r, d_{k+2}, \dots, d_K)$  with  $0 \leq d_m \leq n_m - p_m$  do
16:        $(A', C', \Delta z) \leftarrow \text{COMPUTE\_BLOCK}(k, \vec{p}, A, C, \vec{d})$ 
17:       if result is INVALID then continue
18:       end if
19:        $\vec{p}' \leftarrow (p_{k+2} + d_{k+2}, \dots, p_K + d_K)$ 
20:        $z' \leftarrow z + \Delta z$ 
21:       if  $\nexists (\vec{p}', A', C', \hat{z}) \in \mathcal{Y}^{(k+1)}$  with  $\hat{z} \leq z'$  then
22:         Insert  $(\vec{p}', A', C', z')$  into  $\mathcal{Y}^{(k+1)}$ , pruning dominated states
23:       end if
24:     end for
25:   end for
26: end for
27: // — Extract optimum —
28:  $Z^* \leftarrow \min\{z \mid (\emptyset, A, C, z) \in \mathcal{Y}^{(K)}\}$ 
29: return  $Z^*$ 

```

Remark 4.1 The state pruning in line 17 discards any new state $(\vec{p}', A', C', z')$ for which

a state with the same $(\vec{p}', A', C')$ and a lower or equal z already exists. This is a standard dominance rule that reduces the number of active states without compromising optimality.

5 Complexity Analysis

5.1 State Space Size

For a fixed block index k , the prefix vector $\vec{p} = (p_{k+1}, \dots, p_K)$ can take at most $\prod_{m=k+1}^K (n_m + 1) = O(n^{K-k})$ distinct values. The machine 1 total A ranges over $0, 1, \dots, A_{\max}$ where $A_{\max} = \sum_{j=1}^n a_j$, and the machine 2 completion time C ranges over $0, 1, \dots, C_{\max}$ with $C_{\max} \leq \sum_{j=1}^n (a_j + b_j)$. Hence

$$|\mathcal{Y}^{(k)}| = O(n^{K-k} \cdot A_{\max} \cdot C_{\max}).$$

5.2 Transition Cost

At block k , for a given state we enumerate all selection vectors $\vec{d} = (r, d_{k+2}, \dots, d_K)$, which amounts to $O(n^{K-k-1})$ possibilities. For each $\vec{d}$, Algorithm 2 executes in $O(|T| \log |T|) = O(n \log n)$ time (dominated by Johnson sorting of the selected jobs) followed by an $O(|T|) = O(n)$ pass to compute completion times. Thus each state expansion costs $O(n^{K-k-1} \cdot n \log n)$.

5.3 Overall Complexity

Summing over all K blocks, the total running time is

$$O\left(\sum_{k=0}^{K-1} |\mathcal{Y}^{(k)}| \cdot n^{K-k-1} \cdot n \log n\right) = O(n^{2K-1} \cdot A_{\max} \cdot C_{\max} \cdot \log n).$$

This bound is pseudo-polynomial because $A_{\max}$ and $C_{\max}$ depend on the numeric values of the processing times rather than on n alone. For a fixed number of weight classes K , the complexity is polynomial in n , $A_{\max}$, and $C_{\max}$, which is consistent with the fact that $F2 \parallel \sum W_j C_j$ is NP-hard only in the ordinary sense ?.

5.4 Towards a PTAS

Following the approach of ?, the above exact DP can be converted into a polynomial-time approximation scheme (PTAS) through two rounding steps:

1. **Weight rounding:** Partition the weight range geometrically so that $K = O(\frac{1}{\epsilon} \log W_{\max})$, bounding the number of weight classes.

2. **Time rounding:** Scale and round all processing times a_j, b_j to multiples of $\delta = \varepsilon \cdot \max_j(a_j, b_j)/n$, reducing $A_{\max}$ and $C_{\max}$ to $O(n^2/\varepsilon)$.

With both discretizations in place, the DP runs in time polynomial in n and $1/\varepsilon$, delivering a $(1 + \varepsilon)$ -approximate solution.

6 Conclusion